

```
In [1]: class Person:
        def __init__(gender,age,name,height, weight,blood_gp):
            self.gender = gender
            self.age = age
            self.name = name
            self.height = height
            self.weight = weight
            self.blood_gp = blood_gp
        def eat(self):
            print(f"{self.name} a {self.gender} with height {self.height} and with w
        def sleep(self):
            print(f"{self.name} a {self.gender} with height {self.height} and with w
        def walk(self):
            print(f"{self.name} a {self.gender} with height {self.height} and with w
```

```
In [2]: p1 = Person('Male',20,'Ghaffar', 5.8, 70, 'O+')
```

```
In [4]: p2 = Person('Female',23,'Sara', 5.8, 60, 'O+')
p1 = Person('Male',20,'Ghaffar', 5.8, 70, 'O+')
```

```
In [5]: p1.name
```

```
Out[5]: 'Ghaffar'
```

```
In [6]: p2.name
```

```
Out[6]: 'Sara'
```

```
In [7]: p1.blood_gp
```

```
Out[7]: 'O+'
```

```
In [8]: p2.blood_gp
```

```
Out[8]: 'O+'
```

```
In [9]: p2.eat()
```

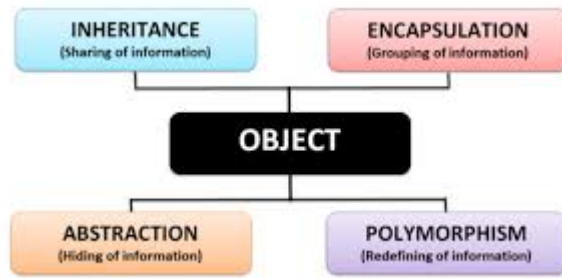
Sara a Female with height 5.8 and with weight 60 is eating food

```
In [10]: p1.eat()
```

Ghaffar a Male with height 5.8 and with weight 70 is eating food

## Four Pillars of Object Oriented Programming

- Encapsulation
- Inheritance
- Polymorphism
- Abstraction



## Encapsulation:

- Binding data with methods

```
In [11]: p1.blood_gp
```

```
Out[11]: 'O+'
```

```
In [12]: p1.blood_gp = "C++"
```

```
In [13]: p1.age
```

```
Out[13]: 20
```

```
In [14]: p2.age
```

```
Out[14]: 23
```

```
In [15]: p2.age=119
```

```
In [24]: class Person:
    def __init__(self,gender,age,name,height, weight,blood_gp):
        # public (can be accessed anywhere)
        self.gender = gender
        self.age = age
        self.name = name
        self.height = height
        self.weight = weight
        self._blood_gp = blood_gp
    def eat(self):
        print(f"{self.name} a {self.gender} with height {self.height} and with w
    def sleep(self):
        print(f"{self.name} a {self.gender} with height {self.height} and with w
    def walk(self):
        print(f"{self.name} a {self.gender} with height {self.height} and with w
    def get_blood_group(self):
        return self._blood_gp
    def set_blood_group(self,new_blood_group):
        blood_groups = ['A+', 'A-', 'B+', 'B-', 'O+', 'O-', 'AB+']
        if new_blood_group in blood_groups:
            self._blood_gp = new_blood_group
        else:
            return "Invalid Blood group"
```

```
In [26]: p2 = Person('Female',23,'Sara', 5.7, 60, 'B+')
p1 = Person('Male',20,'Ghaffar', 5.2, 70, 'O-')
```

# Access Modifiers:

## 1. PublicScope:

Accessible from absolutely anywhere in the program.

Purpose: Used for the main interface (API) of your class. It exposes methods that outside classes need to call to trigger actions or request safe information.

Example: A banking app exposing a public deposit() method.

## 2. PrivateScope:

Accessible only within the exact class where it is declared.

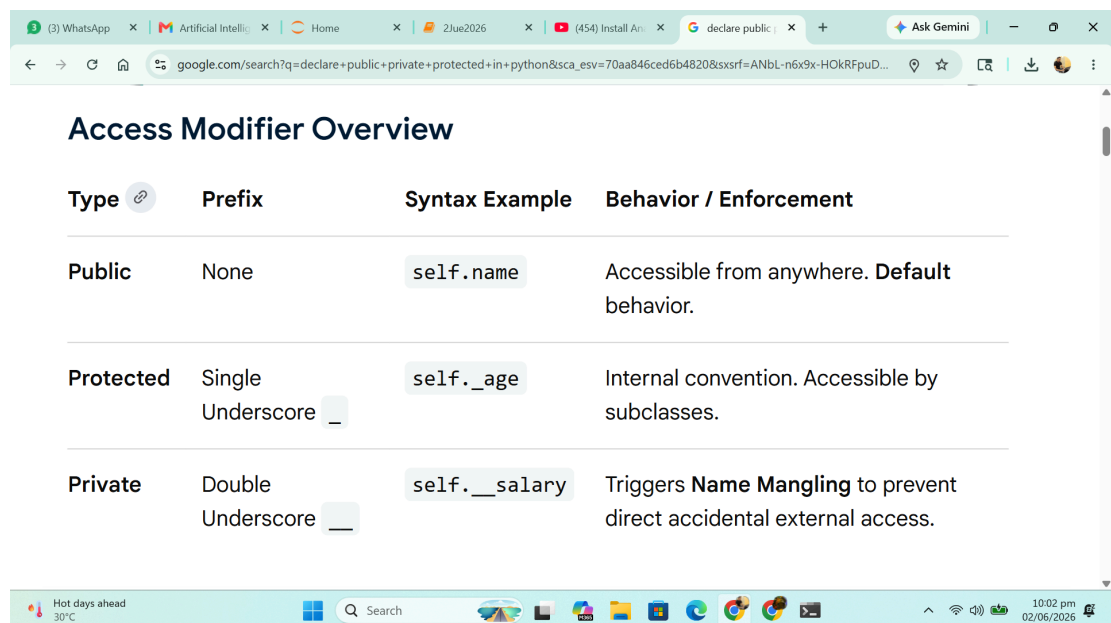
Purpose: This is the most restrictive level and the default setting for many languages. It isolates sensitive fields and complex internal utility functions from outside interference.

Example: A banking app setting the user's accountBalance variable to private so it cannot be directly altered from the outside without validation.

## 3. ProtectedScope:

Accessible within the defining class and by its derived (child) classes.

Purpose: Designed specifically for inheritance. It allows child classes to directly reuse or extend internal attributes while still hiding that data from the rest of the application. Example: A parent class Vehicle has a protected engineType field that a child class ElectricCar can modify.



The screenshot shows a web browser window with multiple tabs. The active tab is titled 'declare public' and shows a search result for 'Access Modifier Overview'. The search bar contains the query 'google.com/search?q=declare+public+private+protected+in+python&scas\_esv=70aa846ced6b4820&sxsrf=ANbL-n6x9x-HOKRFpuD...'. The search result is a table titled 'Access Modifier Overview' with four columns: Type, Prefix, Syntax Example, and Behavior / Enforcement. The table lists three types of access modifiers: Public, Protected, and Private, each with its corresponding prefix, syntax example, and behavior.

| Type      | Prefix                            | Syntax Example             | Behavior / Enforcement                                                      |
|-----------|-----------------------------------|----------------------------|-----------------------------------------------------------------------------|
| Public    | None                              | <code>self.name</code>     | Accessible from anywhere. <b>Default</b> behavior.                          |
| Protected | Single Underscore <code>_</code>  | <code>self._age</code>     | Internal convention. Accessible by subclasses.                              |
| Private   | Double Underscore <code>__</code> | <code>self.__salary</code> | Triggers <b>Name Mangling</b> to prevent direct accidental external access. |

```
In [29]: p1.get_blood_group()
```

Out[29]: 'O-'

In [30]: p1.set\_blood\_group('C++')

Out[30]: 'Invalid Blood group'

In [31]: p1.set\_blood\_group('A+')

In [32]: p1.get\_blood\_group()

Out[32]: 'A+'

## Inheritance:

```
In [33]: class Doctor:
    def __init__(self,gender,name, age, weight,height, blood_group,speciality,hospital,fee):
        self.gender = gender
        self.age = age
        self.name = name
        self.height = height
        self.weight = weight
        self._blood_gp = blood_gp
        self.speciality = speciality
        self.hospital = hospital
        self.fee = fee
    def eat(self):
        print(f"{self.name} a {self.gender} with height {self.height} and with weight {self.weight}")
    def sleep(self):
        print(f"{self.name} a {self.gender} with height {self.height} and with weight {self.weight}")
    def walk(self):
        print(f"{self.name} a {self.gender} with height {self.height} and with weight {self.weight}")
    def get_blood_group(self):
        return self._blood_gp
    def set_blood_group(self,new_blood_group):
        blood_groups = ['A+', 'A-', 'B+', 'B-', 'O+', 'O-', 'AB+']
        if new_blood_group in blood_groups:
            self._blood_gp = new_blood_group
        else:
            return "Invalid Blood group"
    def cure(self):
        print(f"{self.name} a {self.gender} with height {self.height} and with weight {self.weight}")
```

```
In [42]: class Doctor(Person):
    def __init__(self,gender,name, age, weight,height, blood_group,speciality,hospital,fee):
        self.speciality = speciality
        self.hospital = hospital
        self.fee = fee
        super().__init__(gender,name, age, weight,height, blood_group)
    def cure(self):
        print(f"{self.name} a {self.gender} with height {self.height} and with weight {self.weight}")
```

```
In [43]: d1 =Doctor('Female',23,'Sara', 5.7, 60, 'B+', 'eye','abc',1000)
```

```
In [38]: class Student:
        def __init__(self):
            pass
        def __init__(self, name):
            self.name = name
        pass
```

```
In [39]: s1 = Student()
```

```
-----
TypeError                                Traceback (most recent call last)
Cell In[39], line 1
----> 1 s1 = Student()

TypeError: Student.__init__() missing 1 required positional argument: 'name'
```

## IS A relationship:

Doctor IS A person  
Student IS A person  
Dog IS an Animal  
NeemTree is A Tree

## Object as Attribute

```
In [ ]:
```